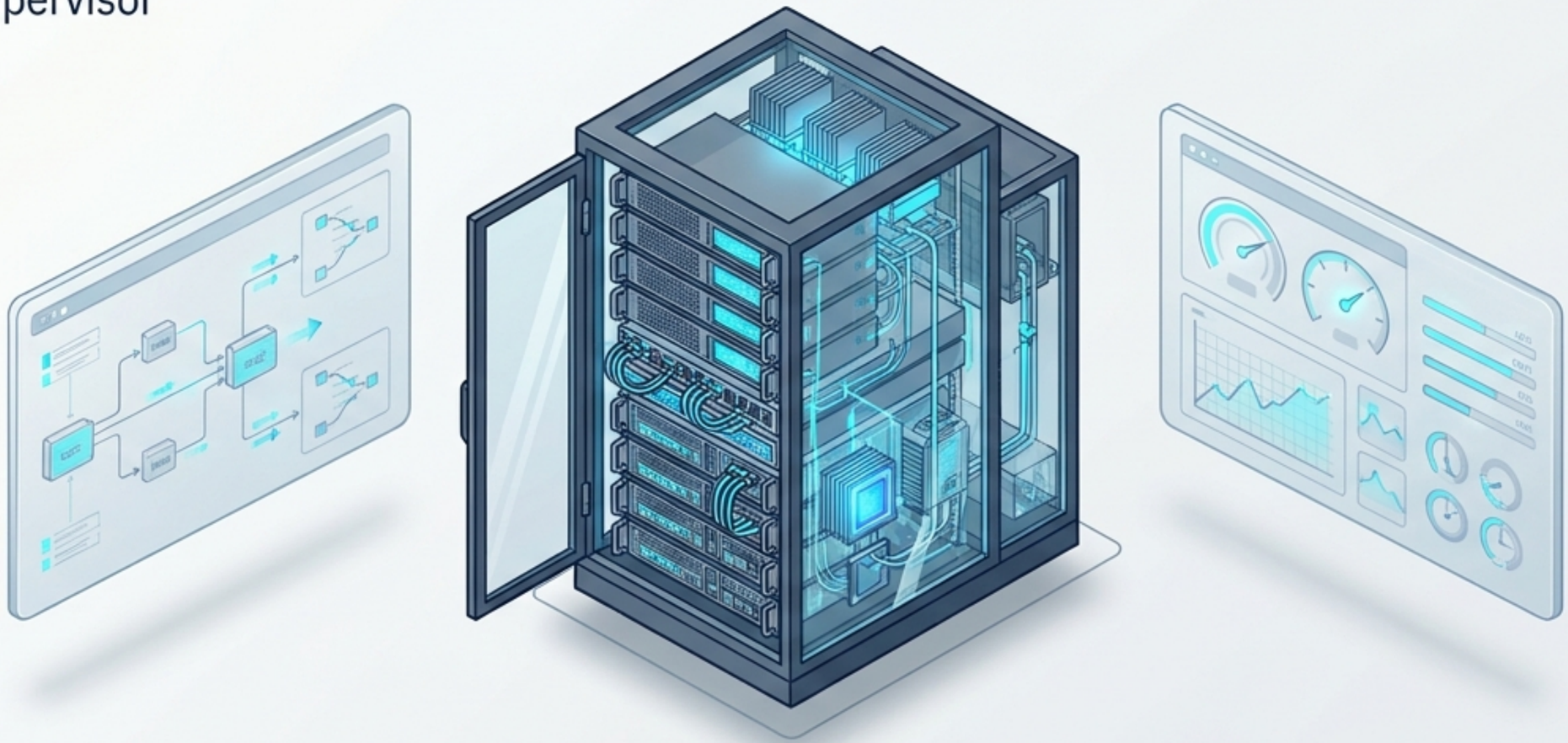
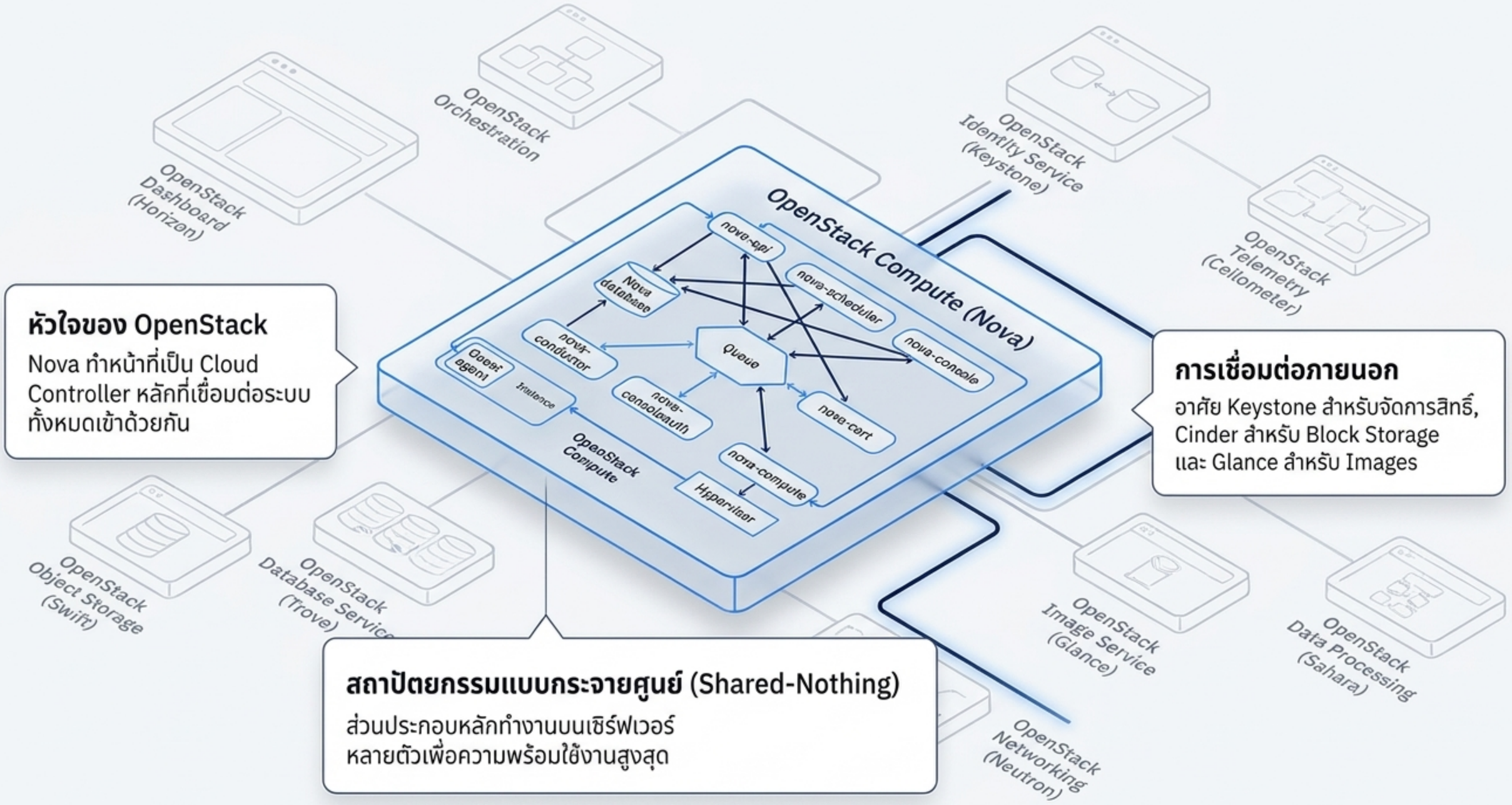
เจาะลึกสถาปัตยกรรม OpenStack Compute (Nova)

กลไกการทำงานเบื้องหลังคลาวด์ระดับองค์กร: การจัดการสิทธิ์, Message Queue, Scheduler และ Hypervisor



Gemini Notebook



หัวใจของ OpenStack

Nova ทำหน้าที่เป็น Cloud Controller หลักที่เชื่อมต่อระบบทั้งหมดเข้าด้วยกัน

การเชื่อมต่อภายนอก

อาศัย Keystone สำหรับจัดการสิทธิ์, Cinder สำหรับ Block Storage และ Glance สำหรับ Images

สถาปัตยกรรมแบบกระจายศูนย์ (Shared-Nothing)

ส่วนประกอบหลักทำงานบนเซิร์ฟเวอร์หลายตัวเพื่อความพร้อมใช้งานสูงสุด

Gemini Notebook

กายวิภาคของ Nova (4 องค์ประกอบหลัก)

API Server (Cloud Controller)

ด่านหน้า (Web Services Front End) รับคำสั่งผ่าน HTTP รันระบบยืนยันตัวตน และตรวจสอบสิทธิ์ของผู้ใช้



Message Queue (AMQP)

ระบบท่อส่งข้อมูลส่วนกลางที่ประสานงานระหว่างโหนดและเซอร์วิสต่างๆ โดยไม่ทำให้ระบบชะงัก

Nova Scheduler

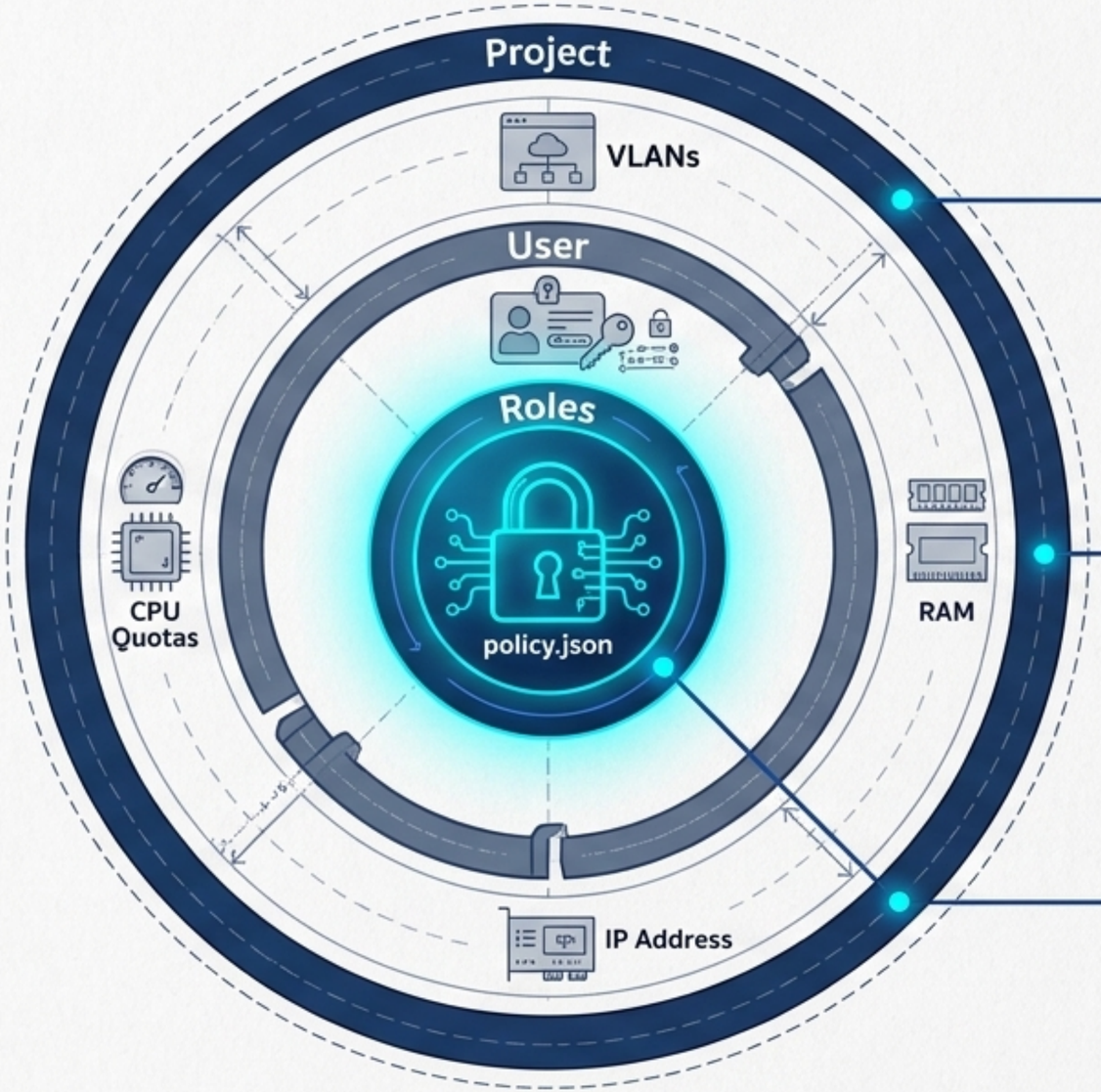
สมองที่ตัดสินใจและคัดเลือก Compute Controller ที่เหมาะสมที่สุดในการโฮสต์ Instance

Compute Worker

กล้ามเนื้อที่รับคำสั่งไปสั่งการ Hypervisor เพื่อสร้าง, ลบ, หรือรีบูต Instance บนเครื่องโฮสต์

Gemini Notebook

ปราการด่านแรก:การจัดการสิทธิ์ด้วย RBAC



Project (Tenant)

คอนเทนเนอร์แยกทรัพยากร (Isolated Resource Containers)
ควบคุมโควต้า (เช่น จำนวน Volume, Core ของ CPU, RAM และ IP Address)

User

ผู้ใช้งานที่มีรหัสผ่านและ Keypairs เฉพาะตัว
ระบบจะผูก User เข้ากับ Project อัตโนมัติหากไม่ระบุใน API

Roles (policy.json)

กำหนดสิทธิ์การดำเนินการของ User เช่น ต้องมีสิทธิ์เป็น Admin เท่านั้นถึงจะสามารถจัดสรร Public IP Address ได้

Gemini Notebook

ระบบประสาทส่วนกลาง: การสื่อสารผ่าน Message Queue

แบบเดิม (Synchronous HTTP)

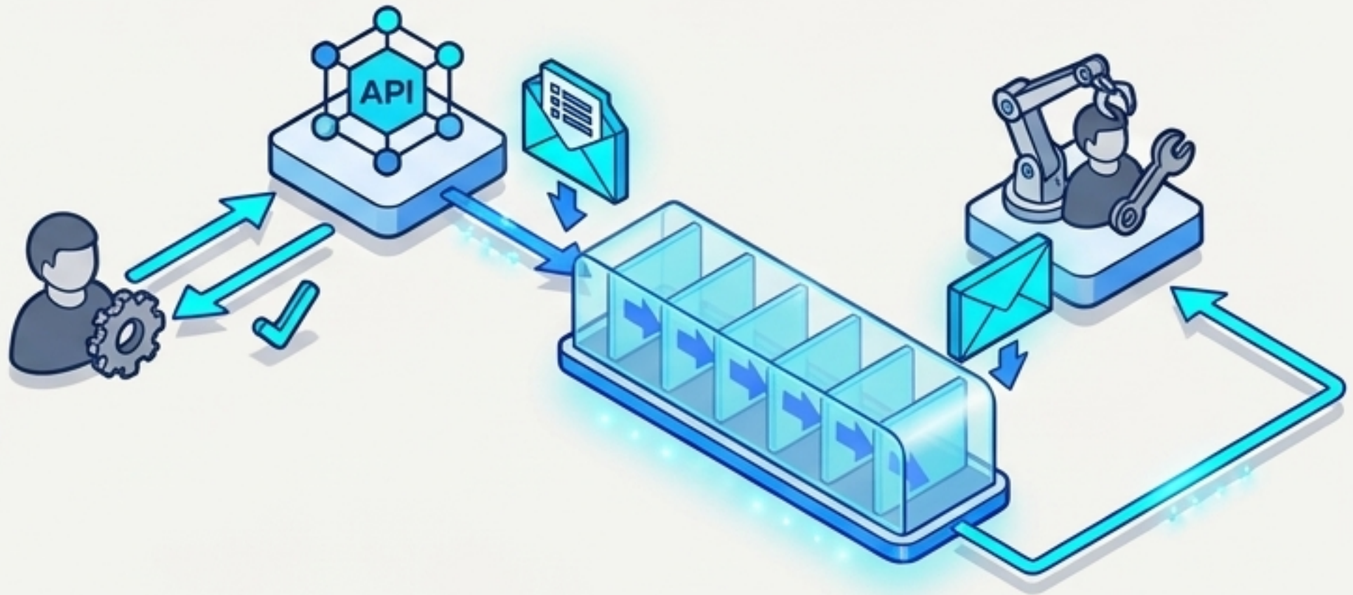
ระบบถูกบล็อก (Block) ระหว่างรอการประมวลผลคลาวด์ที่ใช้เวลานาน



แบบ OpenStack (Asynchronous AMQP)

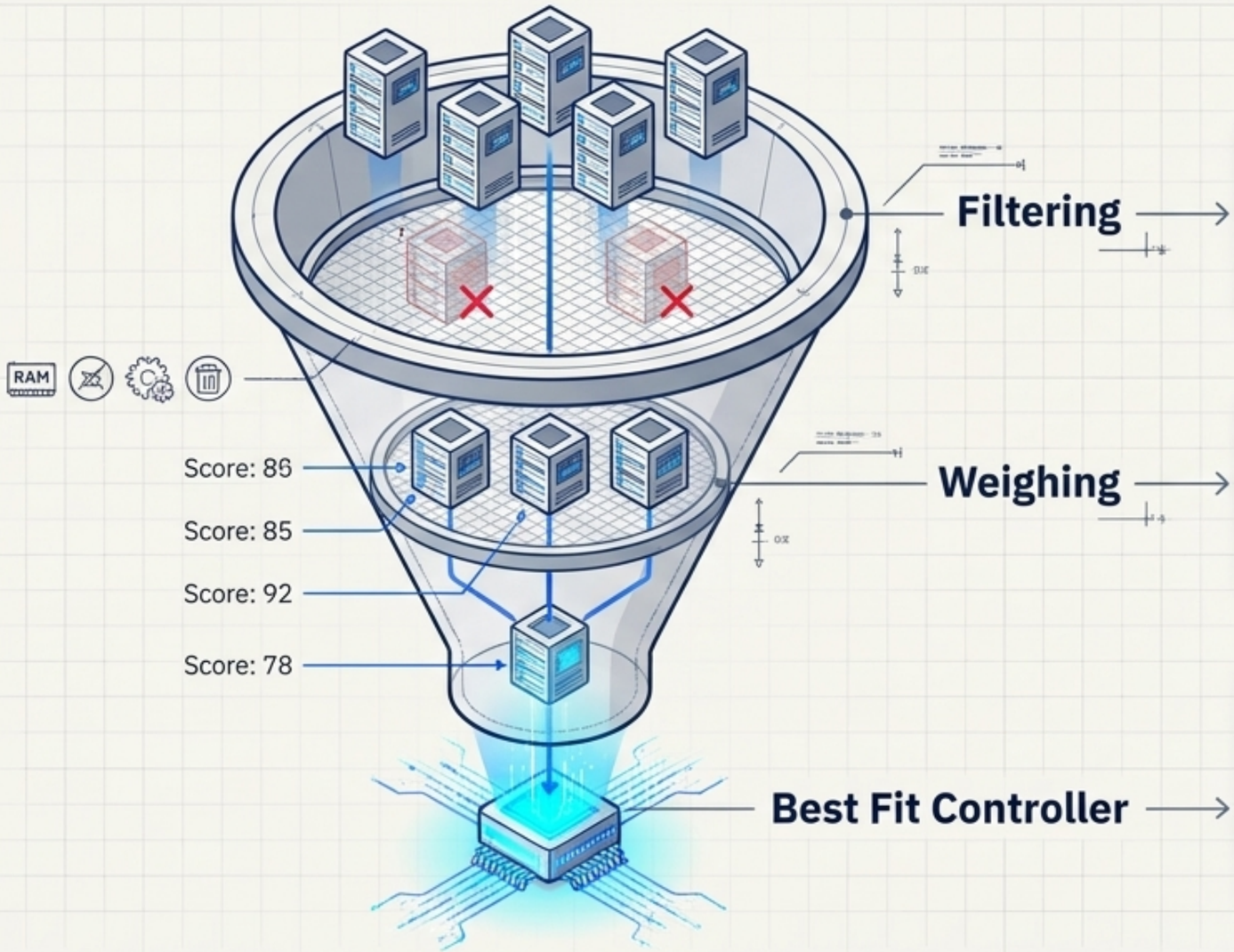
ใช้ Advanced Message Queuing Protocol API ส่งคำสั่งเข้า Queue แล้วแจ้งผลกลับทันที

Worker จะมารับงานและใช้ Callback แจ้งเมื่อทำงานเสร็จ ป้องกันปัญหาระบบค้าง (Bottleneck)



Gemini Notebook

สมองสั่งการ: หลักการทำงานของ Nova Scheduler



กระบวนการคัดกรอง (Filtering)

ตัด Compute Node ที่ไม่ตรงตามเงื่อนไขออก เช่น RAM ไม่พอ, OS ไม่ตรง, หรือโควต้าเต็ม

กระบวนการให้คะแนน (Weighing)

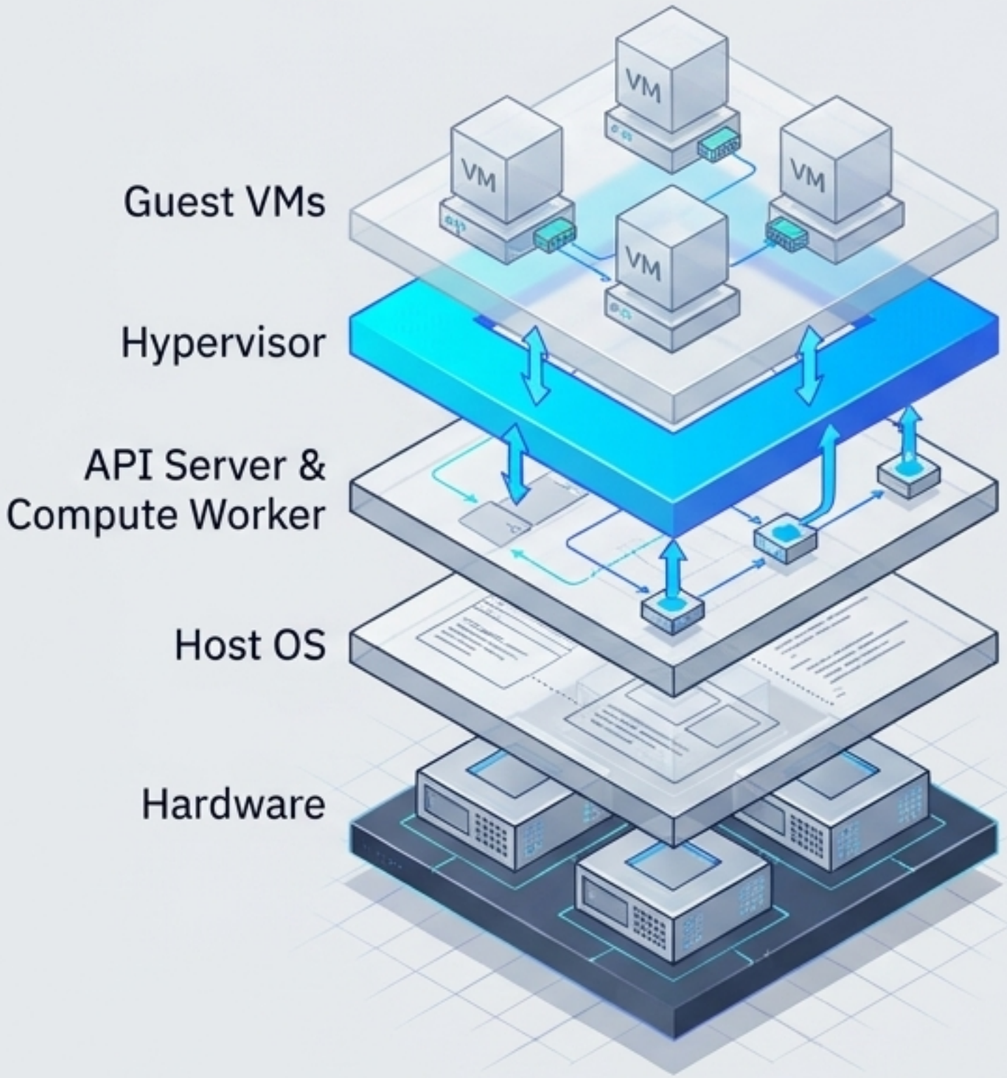
ประเมิน Node ที่เหลืออยู่ เช่น Node ไหนมีโหลดการทำงานต่ำที่สุด หรือมีทรัพยากรว่างมากที่สุด

ผลลัพธ์ (Best Fit)

ได้ Compute Controller ที่เหมาะสมที่สุดในการโฮสต์ Instance เพื่อกระจายภาระงาน (Load Balancing) อย่างมีประสิทธิภาพ

Gemini Notebook

กล้ามเนื้อผู้ลงมือทำ: ชั้นของการรัน Hypervisor



การเชื่อมต่อ

Compute Worker จะควบคุม Hypervisors ผ่าน API Server

ความยืดหยุ่น

OpenStack เป็น Hypervisor-agnostic สามารถจัดการคลาวด์ด้วย Hypervisor หลายประเภทใน Availability Zones ที่ต่างกันได้

มาตรฐานอุตสาหกรรม

การพัฒนาส่วนใหญ่ของ OpenStack ปัจจุบันเน้นไปที่ระบบที่ใช้ KVM และ Xen-based เป็นหลัก

Gemini Notebook

ระบบนิเวศของ Hypervisor ที่รองรับ



Full Virtualization

ตัวอย่าง: KVM, QEMU, VMware vSphere, Hyper-V, Xen

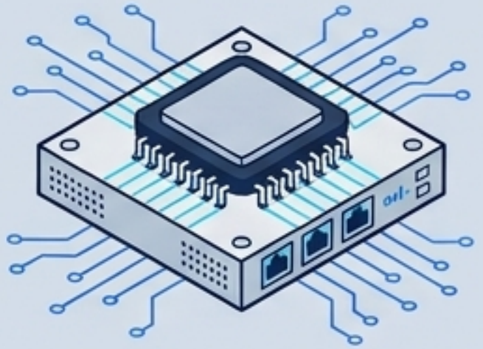
จุดเด่น: จำลองฮาร์ดแวร์เต็มรูปแบบ แยกส่วน (Isolation) สูง เหมาะสำหรับ Enterprise VMs มาตรฐาน



Containerization

ตัวอย่าง: Docker, Linux Containers (LXC)

จุดเด่น: น้ำหนักเบา (Lightweight) บูตเร็ว ใช้ทรัพยากรร่วมกับ Host OS



Baremetal & Others

ตัวอย่าง: Ironic (Baremetal), User Mode Linux (UML)

จุดเด่น: ประสิทธิภาพสูงสุดระดับฮาร์ดแวร์โดยไม่มีชั้น Hypervisor คั่น

Gemini Notebook

ชิ้นส่วนตั้งต้น: Images และ Flavors



Ubuntu/Windows

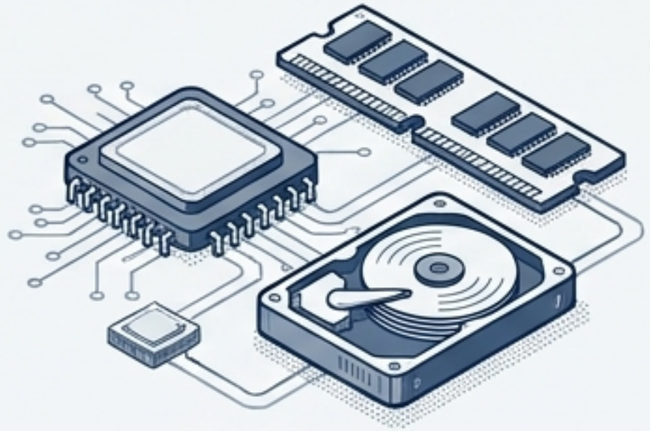


Image (ระบบปฏิบัติการ)

- เทมเพลตเริ่มต้น ปกติเก็บไว้ใน OpenStack Image Service (Glance)
- คำสั่ง **openstack image list** แสดง ID, Name, และ Status

Flavor (เทมเพลตฮาร์ดแวร์)

- กำหนดขนาดทรัพยากร (RAM, Disk, Ephemeral, VCPUs) ที่จะจอง
- คำสั่ง **openstack flavor list** (เช่น m1.tiny, m1.large) ปกติจัดการโดย Admin



Instance

Gemini Notebook

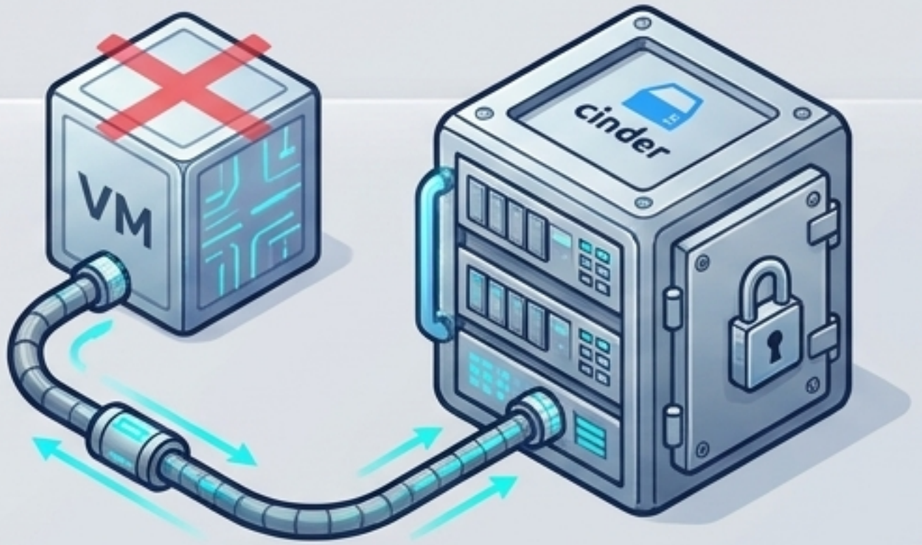
บริบทของ Storage: Ephemeral vs Persistent

Ephemeral Storage (พื้นที่ชั่วคราว)



- ผูกติดกับอายุขัยของ Instance ข้อมูลจะหายไปเมื่อลบ Instance
- ขนาดถูกกำหนดโดย Flavor (ยกเว้น m1.tiny)

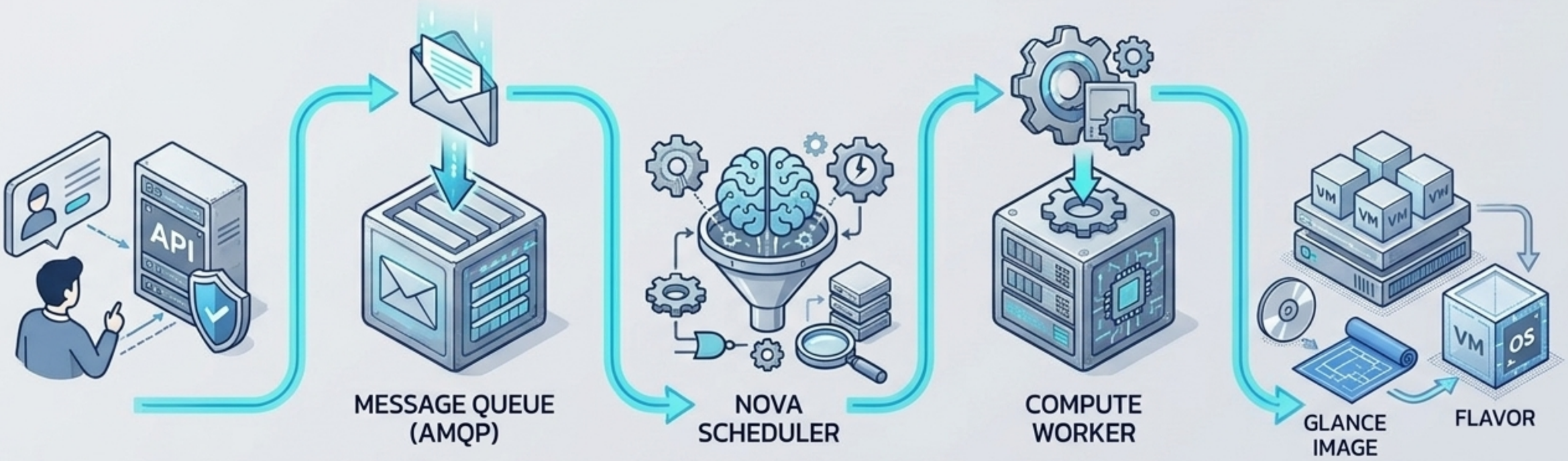
Persistent Volume (พื้นที่ถาวร)



- ให้บริการโดย OpenStack Block Storage (Cinder)
- ข้อมูลยังอยู่แม้ลบ Instance เข้าถึงได้ครั้งละ 1 Instance
- สามารถตั้งค่าให้บูตจาก Persistent Volume ได้เลย

Gemini Notebook

วงจรชีวิตของคำสั่งสร้าง Instance (Synthesis Lifecycle)



1. ยืนยันตัวตน:
User ส่งคำสั่งผ่าน API Server ระบบตรวจสอบสิทธิ์ (RBAC/Keystone)

2. ส่งคำสั่งเข้าคิว:
API ส่งคำสั่งสร้าง Instance ไปยัง Message Queue (AMQP) แบบ Asynchronous

3. ค้นหาผู้รับเหมา:
Nova Scheduler อ่านคิว ประเมิน Filter/Weight แล้วเลือก Compute Node ที่ดีที่สุด

4. สั่งการและลงมือ:
Compute Worker บนโหนดที่ถูกเลือกรับคำสั่งจาก Queue

5. ประกอบร่าง:
Hypervisor ดึง Image (จาก Glance) และตั้งค่าตาม Flavor เพื่อบูต VM ขึ้นทำงาน

Gemini Notebook

บทสรุปสถาปัตยกรรม OpenStack Compute



ความปลอดภัยรัดกุม (Governance)

ระบบ RBAC ควบคุมโควต้าและสิทธิ์ระดับ Project/User อย่างเคร่งครัด



ไม่เกิดคอขวด (Asynchronous)

สถาปัตยกรรม Shared-Nothing สื่อสารผ่าน AMQP ช่วยให้ระบบตอบสนองได้ตลอดเวลา



จัดสรรทรัพยากรชาญฉลาด

Nova Scheduler กระจายโหลด (Load Balancing) อัตโนมัติด้วยอัลกอริทึม Filter และ Weight



ความยืดหยุ่นสูง (Hypervisor Agnostic)

ไม่ผูกขาดเทคโนโลยี รองรับตั้งแต่ KVM, Docker ไปจนถึง Baremetal ภายใต้ API เดียวกัน

Gemini Notebook